Problema de Coloreado de Grafos con Pesos y Colores Preasignados

Algoritmos Evolutivos

Jerónimo Gil

Universidad EAFIT
Curso: Heurística
Profesor: Juan Carlos Rivera Agudelo

8 de Noviembre de 2026

Índice

1. Definición del problema
2. Métodos Constructivos y Aleatorizados (Primera Entrega)
3. Búsqueda en Vecindarios (Segunda Entrega)
4. Metaheuristicos (Tercera Entrega)
5. Algoritmo Genético
6. Algoritmo Evolutivo Híbrido (Genético + VND)
7. Resultados
8. Comparación métodos implementados
9. Comparación métodos de las entregas previas
10. Validación
11. Conclusiones
12. Referencias

Definición del problema (GCP con pesos y colores preasignados)

Descripción

Dado un grafo $G = (V, E)$ con n vértices, pesos asociados w_i, y m nodos con color preasignado, el objetivo es asignar una etiqueta de color a cada nodo sin causar conflictos entre vecinos.

Objetivos de Optimización:

- z_1: Minimizar el número total de colores utilizados.
- z_2: Minimizar la diferencia entre el peso máximo y mínimo asignado a un color:

$$z_2 = \max_c W_c - \min_c W_c$$

Función Objetivo Combinada:

$$f(S) = 100 \cdot \left(z_1 + \frac{z_2}{\sum_{i \in V} w_i} \right)$$

- Se penaliza fuertemente el uso de colores adicionales (z_1).
- Se desempata buscando equilibrio en la suma de pesos (z_2).

Métodos Constructivos y Aleatorizados (Primera Entrega)

Constructivo

1. Priorizar nodos por: saturación, peso y grado.
2. Asignar el color factible con menor peso acumulado W_c.
3. Abrir un nuevo color si es necesario.

GRASP ($\alpha = 0.1$, n = 50)

1. Evaluar candidatos mediante saturación y grado.
2. Construir la lista de candidatos
3. Seleccionar aleatoriamente un candidato.
4. Repetir n_{sol} veces y conservar la mejor solución.

Ruido (r = 0.25, n = 50)

1. Perturbar los pesos
2. Calcular el score
3. Seleccionar el mayor score.
4. Generar n_{sol} soluciones y conservar la mejor.

Idea común

Los tres métodos generan soluciones factibles respetando las preasignaciones; se diferencian en el criterio de selección de nodos y en el nivel de aleatorización.

Búsqueda en Vecindarios (Segunda Entrega)

N_1: 1-Move
Cambia el color de **un único nodo** no preasignado de la solución actual.

$$c_i \longrightarrow c_j$$

- Adopta un color ya existente.
- Se valida la factibilidad.

Objetivo: Redistribuir pesos individualmente para afinar el balance y reducir Z_2.

N_2: Swap (Intercambio)
Intercambia las etiquetas de color entre **dos nodos** no preasignados.

$$(c_i, c_j) \longrightarrow (c_j, c_i)$$

- Los nodos deben tener distinto color y se intercambian simultaneamente
- Se verifica la factibilidad.

Objetivo: Redistribuir los pesos asociados.

N_3: Recoloración
Selecciona **dos nodos** que comparten color y los modifica simultáneamente.

$$(c_i, c_i) \longrightarrow (c_j, c_k)$$

- Parten del mismo color y adoptan colores nuevos (iguales o distintos).
- Se valida la factibilidad.

Objetivo: Generar cambios estructurales drásticos, ideal para vaciar colores y reducir Z_1.

Variable Neighborhood Descent (VND)

Se construyó un método que encadena tres vecindarios (N_1, N_2 y N_3) para escapar de óptimos locales.

- La búsqueda alterna entre los vecindarios de 1-Move, Swap y Recoloración.
- Si se logra una mejora en la función objetivo, el algoritmo regresa inmediatamente al vecindario más rápido (N_1).
- El ciclo de búsqueda local finaliza únicamente cuando los tres vecindarios fallan de manera consecutiva.

Metaheurística Híbrida

Se diseñó un método que combina tres elementos avanzados para evitar el estancamiento.

- Múltiples soluciones iniciales
- Perturbación: Se altera aleatoriamente el color de un porcentaje de nodos para forzar al algoritmo a explorar nuevas soluciones.
- Recocido Simulado: Mecanismo de enfriamiento que acepta soluciones peores.

Algoritmo Genético

1. Inicialización

- **Población Inicial:** Se genera utilizando el algoritmo constructivo con Ruido ($r = 0{,}25$) para garantizar diversidad y partir de soluciones base de alta calidad.

2. Selección

- **Elitismo:** El mejor individuo de la generación actual pasa intacto a la siguiente generación para no perder el óptimo histórico.
- **Torneo ($k = 2$):** Para el resto de la población, se seleccionan 2 individuos al azar y el que tenga mejor función objetivo se convierte en padre.

3. Reproducción (Cruce y Mutación)

- **Cruce por un Punto:** Se elige un punto de corte aleatorio. El hijo hereda la primera mitad del Padre 1 y la segunda del Padre 2. Se fuerzan los colores de los nodos preasignados.
- **Mutación Uniforme:** Con una probabilidad dada, se cambia el color de un nodo (no preasignado) por otro color disponible al azar.
- **Reparación:** Tras el cruce y mutación, se aplica una función para asegurar que las etiquetas sean consecutivas y factibles.

Algoritmo Genético: Pseudocódigo

```
Entrada: instancia, tam_poblacion, generaciones, prob_mutacion, tiempo_max
Salida: mejor_solucion_global

t_inicio = TiempoActual()
Poblacion = []
Para i = 1 Hasta tam_poblacion Hacer // 1. Inicialización
    Si TiempoTranscurrido() > tiempo_max Romper
    Poblacion.Añadir( AlgoritmoRuido(instancia, r=0.25) )
Fin Para

Para gen = 1 Hasta generaciones Hacer // 2. Ciclo Evolutivo
    Si TiempoTranscurrido() > tiempo_max Romper
    Evaluar(Poblacion)
Actualizar(mejor_solucion_global)

    Nueva_Poblacion =  [MejorIndividuo(Poblacion)]     // 3. Elitismo
    Mientras Tamaño(Nueva_Poblacion) < tam_poblacion Hacer  // 4. Cruce y Mutación
        Si TiempoTranscurrido() > tiempo_max Romper
        Padre1 = SeleccionTorneo(Poblacion, k=2)
        Padre2 = SeleccionTorneo(Poblacion, k=2)
        Hijo = CrucePorUnPunto(Padre1, Padre2)
        Hijo = MutacionAleatoria(Hijo, prob_mutacion)
        Nueva_Poblacion.Añadir(Hijo)
    Fin Mientras
    Poblacion = Nueva_Poblacion
Fin Para
Retornar mejor_solucion_global
```

Algoritmo Genético: Análisis de Parámetros

Metodología del Diseño de Experimentos

La calibración paramétrica se ejecutó sobre un subconjunto de instancias, para que no tomará varios días de cómputo, las instancias seleccionadas fueron: GCP1, GCP4, GCP7, GCP10, GCP13, GCP16, GCP19.

- **El valor de la diversidad:** Las poblaciones más grandes (100 y 80 individuos) logran los mejores resultados (GAP de 7.46 %). Una mayor población evita que el algoritmo se estanque rápidamente.
- **Mutación Alta vs Baja:** Con 100 individuos, una mutación agresiva (0,3) funciona mejor al. Con 80 individuos, una mutación baja (0,1) es suficiente para mantener el equilibrio.

Mejor Combinación

Pob = 100 y Mut = 0.3.

Cuadro: Mejores Configuraciones - Genético

Pob.	Mut.	GAP Prom (%)	Tiempo (s)
100	0.3	**7.4646**	19.77
80	0.1	7.4647	12.99
50	0.1	7.4920	**7.57**
100	0.1	7.5404	15.78
100	0.2	7.5404	17.78
80	0.3	7.6279	16.87

Evolutivo Híbrido

1. Población Inicial de Élite

- **Diversificación:** Las soluciones base se generan con el algoritmo de Ruido ($r = 0{,}25$).
- **Intensificación temprana:** Inmediatamente después de nacer, cada solución es sometida a Búsqueda Local (VND).

2. Selección por Torneo

- **Elitismo:** El mejor individuo de la generación pasa intacto a la siguiente.
- **Torneo ($k = 3$):** Compiten 3 individuos elegidos al azar; el de mejor función objetivo se convierte en padre.

3. Reproducción e Hibridación

- **Cruce y Mutación:** Se aplica el cruce por un punto y una mutación aleatoria para mezclar información y explorar nuevas soluciones.
- **Hibrido (VND):** Al cruzarse, los hijos suelen perder su condición de óptimo local. Antes de ingresar a la nueva generación, se aplica VND a cada hijo.

Evolutivo Híbrido : Pseudocódigo

```
Entrada: instancia, vecindarios, tam_poblacion, generaciones, tiempo_max
Salida: mejor_solucion_global

t_inicio = TiempoActual()
Poblacion = []
Para i = 1 Hasta tam_poblacion Hacer // 1. Población Inicial
    Si TiempoTranscurrido() > tiempo_max Romper
    Sol_Base = AlgoritmoRuido(instancia, r=0.25)
    Sol_Opt = VND(Sol_Base, vecindarios, tiempo_restante) // Intensificación inicial
    Poblacion.Añadir(Sol_Opt)
Fin Para

Para gen = 1 Hasta generaciones Hacer // 2. Ciclo Evolutivo
    Si TiempoTranscurrido() > tiempo_max Romper
    Evaluar(Poblacion) y Actualizar(mejor_solucion_global)

    Nueva_Poblacion = [MejorIndividuo(Poblacion)]     // 3. Elitismo
    Mientras Tamaño(Nueva_Poblacion) < tam_poblacion Hacer  // 4. Reproducción e Hibridación
        Si TiempoTranscurrido() > tiempo_max Romper
        Padre1 = SeleccionTorneo(Poblacion, k=3)
        Padre2 = SeleccionTorneo(Poblacion, k=3)
        Hijo = CrucePorUnPunto(Padre1, Padre2)
        Hijo = Mutacion(Hijo, prob_mutacion)
        Hijo_Opt = VND(Hijo, vecindarios, tiempo_restante) // VND a cada hijo
        Nueva_Poblacion.Añadir(Hijo_Opt)
    Fin Mientras
    Poblacion = Nueva_Poblacion
Fin Para
Retornar mejor_solucion_global
```

Evolutivo Híbrido: Análisis de Parámetros

Metodología del Diseño de Experimentos

La calibración paramétrica se ejecutó sobre un subconjunto de instancias, para que no tomará varios días de cómputo, las intancias seleccionadas fueron: GCP1, GCP4, GCP7, GCP10, GCP13, GCP16, GCP19.

Análisis de Parámetros: Evolutivo Híbrido

- **Dominancia Poblacional:** El tamaño de la población es el factor más crítico para la calidad. Las configuraciones con 20 individuos alcanzan el mejor GAP, mientras que reducir la población a 10 penaliza severamente el GAP.
- **Neutralización de la Mutación:** Para una población de 20, variar la probabilidad de mutación produce resultados de GAP idénticos. Esto indica que el poder de intensificación del VND es tan profundo que repara y homogeniza las perturbaciones genéticas estructurales sin importar su intensidad.

Cuadro: Mejores Configuraciones - Hibrido

Pob.	Mut.	GAP (%)	Tiempo (s)
20	0.1	**7.0106**	560.22
20	0.2	**7.0106**	579.24
20	0.3	**7.0106**	577.90
15	0.2	7.0135	400.46
10	0.1	7.5593	**252.11**

Mejor Combinación

Pob: 20 y Mut: 0.1

Resultados

GAP

Instancia	Genético Evolutivo	Híbrido
GCP_1	0.00	0.00
GCP_2	0.00	0.00
GCP_3	0.00	0.00
GCP_4	0.04	0.00
GCP_5	0.04	0.01
GCP_6	0.11	0.10
GCP_7	2.61	0.00
GCP_8	2.63	0.04
GCP_9	0.59	0.02
GCP_10	0.53	0.02
GCP_11	0.84	0.02
GCP_12	0.73	0.02
GCP_13	17.78	17.62
GCP_14	0.20	0.00
GCP_15	7.84	7.70
GCP_16	7.87	7.72
GCP_17	3.85	0.00
GCP_18	3.67	0.00
GCP_19	22.78	26.73
Promedio	**3.80**	**3.16**

Tiempo de Cómputo

Instancia	Genético Evolutivo	Híbrido
GCP_1	3600.0000	3600.0000
GCP_2	3600.0000	3600.0000
GCP_3	3600.0000	3600.0000
GCP_4	3600.0000	3600.0700
GCP_5	3600.0000	3600.0100
GCP_6	3600.0000	3600.0300
GCP_7	3600.0000	3600.0200
GCP_8	3600.0000	3600.0100
GCP_9	3600.0000	3600.0500
GCP_10	3600.0000	3600.1100
GCP_11	3600.0000	3600.2700
GCP_12	3600.0000	3600.2200
GCP_13	3600.0000	3600.0500
GCP_14	3600.0100	3600.1100
GCP_15	3600.0000	3600.1900
GCP_16	3600.0000	3600.1700
GCP_17	3600.0000	3603.7400
GCP_18	3600.0000	3601.3200
GCP_19	3600.0500	3600.0800
Promedio	**3600.0032**	**3600.3395**

Comparación métodos implementados

Cuadro: Comparación global entre el algoritmo Genético y el Híbrido

	Genético	**Híbrido**
GAP promedio (%)	3.7953	3.1586
Tiempo promedio (s)	3600.0032	3600.3395

Comparación métodos implementados - GAP

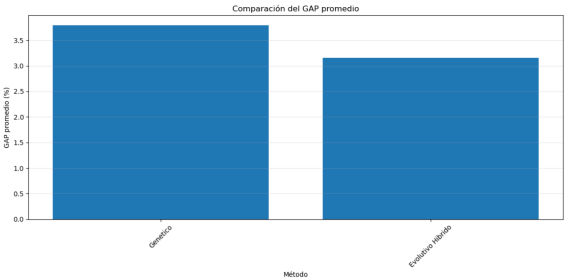


Figura: GAP (%) por método.

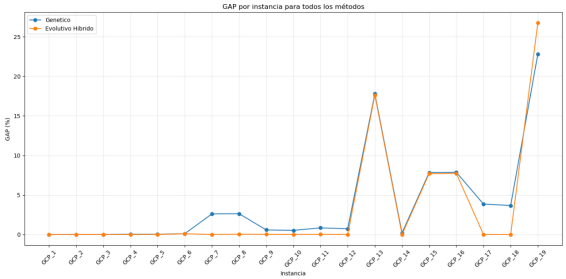


Figura: GAP (%) por instancia.

Comparación métodos implementados - Tiempos de Computo

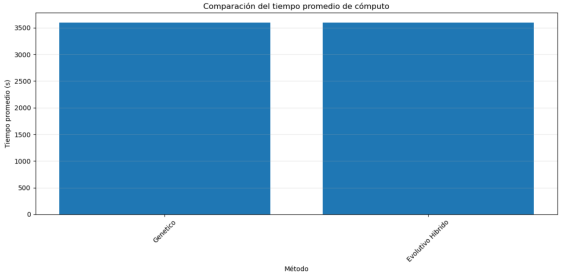


Figura: Tiempo (s) por método.

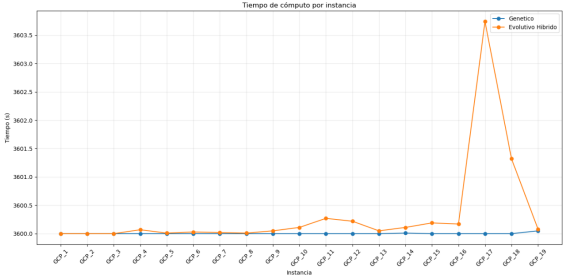


Figura: Tiempo (s) por instancia.

Comparación métodos implementados - Análisis

- Ambos algoritmos se ajustan de manera estricta y exitosa al límite de 1 hora exigido, promediando $\approx 3600\,\text{s}$ de ejecución de forma constante. Las desviaciones marginales del Híbrido (ej. 3603,7 s en `GCP_17`) obedecen al cierre de ciclos locales del VND antes de evaluar el reloj global.
- La integración de la Búsqueda Local (VND) como mecanismo de reparación rinde frutos evidentes, disminuyendo el GAP promedio de un 3,80 % en el Genético a un 3,16 % en el Híbrido.
- El enfoque Híbrido obtiene un GAP de prácticamente 0,00 % en la gran mayoría de las instancias base, corrigiendo las deficiencias que el Genético no logra pulir por sí solo (como en `GCP_7` con 2,61 % o `GCP_17` con 3,85 %).
- De manera contraintuitiva, el Genético supera al Híbrido en la instancia más pesada (`GCP_19`), registrando un GAP de 22,78 % frente al 26,73 % del hibrido. Al aplicar el VND a cada hijo en una instancia tan colosal, el Híbrido agota los 3600 segundos iterando muy pocas generaciones, estancándose antes de lograr una convergencia óptima.

Comparación métodos de las entregas previas

Método	**GAP promedio (%)**	**Tiempo promedio (s)**
Constructivo	12.2395	2.1189
GRASP	10.5180	4.0900
Ruido	4.7972	0.0132
FI_N1	3.4398	0.4116
BI_N1	3.4931	14.5816
FI_N2	4.5238	1.8747
BI_N2	4.4546	3.7500
FI_N3	4.2786	0.2374
BI_N3	4.4006	1.8968
VND	3.1501	10.4574
Hibrido_ILS_SA	2.9852	49.9268
Genético	3.7953	3600.0032
Evolutivo Híbrido	3.1586	3600.3395

Comparación métodos de las entregas previas - GAP

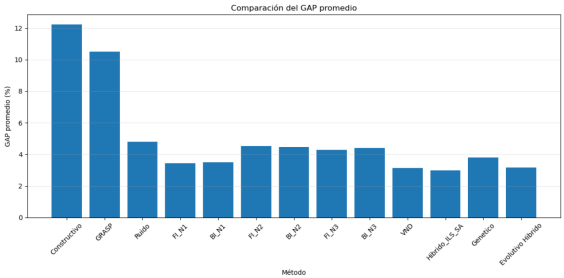


Figura: GAP (%) por método.

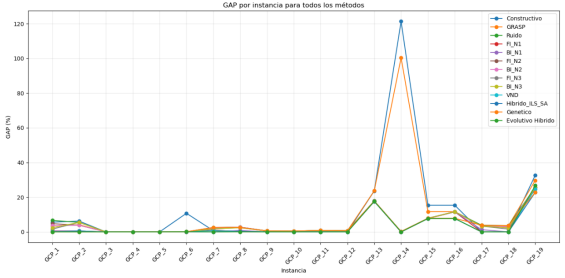


Figura: GAP (%) por instancia.

Comparación métodos de las entregas previas - Tiempos de Computo

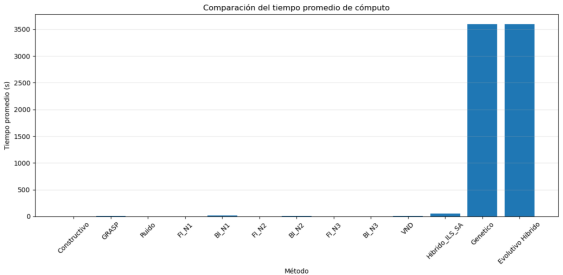


Figura: Tiempo (s) por método.

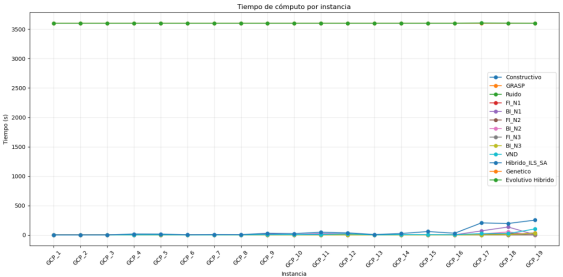


Figura: Tiempo (s) por instancia.

Comparación métodos de las entregas previas - Análisis

- La progresión de la calidad es evidente. Los heurísticos base promedian GAPs superiores al 10,5 % (Constructivo y GRASP). La Búsqueda Local (FI_N1) reduce este margen al 3,44 %, mientras que los metaheurísticos avanzadas (Híbrido ILS-SA, Evolutivo Híbrido) disminuye el GAP a la frontera del $2{,}9\,\% - 3{,}1\,\%$.
- El Híbrido ILS-SA de la entrega anterior logró el mejor GAP global (2,98 %) consumiendo apenas ≈ 50 s promedio. En contraste, los enfoques poblacionales (Genético y Evolutivo Híbrido) agotan la hora completa (3600 s) para alcanzar GAPs similares (3,80 % y 3,16 %), demostrando rendimientos no tan buenos frente a el aumento considerable de tiempo.
- El caso de la instancia `GCP_14`, que disparaba el GAP del Constructivo al 121,33 % y de GRASP al 100,33 %, es resuelto por el Evolutivo Híbrido y el Híbrido ILS-SA, que la llevan al 0,00 %.
- Las instancias `GCP_13` y `GCP_19` se mantienen como el límite del problema. Ningún método, ni siquiera con 3600 s, logra bajar de un $\approx 17{,}6\,\%$ para `GCP_13` ni de $\approx 22{,}7\,\%$ para `GCP_19`.

Validación de Factibilidad

Criterios de Validación

Cada solución generada se somete a una validación estricta:

- Todos los nodos deben tener una etiqueta de color entera.
- Las etiquetas utilizadas deben ser consecutivas y comenzar en 0.
- No existen dos nodos adyacentes con el mismo color.
- Se conservan los colores de los nodos preasignados.
- Ningún nodo queda sin colorear.

Resultado de la Validación

Se evaluaron las soluciones obtenidas por los métodos: Constructivo, GRASP, Ruido, FI-N_1, FI-N_2, FI-N_3, BI-N_1, BI-N_2, BI-N_3, VND, Metaheuristica Hibrida, Genético y Evolutivo Hibrido en las 19 instancias y se obtuvieron 247/247 **soluciones válidas** (100 % de factibilidad). Así, todos los métodos produjeron soluciones que cumplen las restricciones de coloración y las condiciones de preasignación establecidas para las 19 instancias.

Conclusiones

- La trayectoria metodológica desde heurísticos constructivas básicas (GAP promedio $\approx 12{,}2\,\%$) hasta metaheurísticos complejos logró reducir el GAP al $2{,}9\,\% - 3{,}1\,\%$, demostrando la necesidad de mecanismos de diversificación e intensificación.
- El Híbrido ILS-SA se posiciona como el punto óptimo del proyecto, obteniendo el mejor GAP global ($2{,}99\,\%$) en apenas $\approx 50\,\text{s}$. Por su parte, el Evolutivo Híbrido alcanza una precisión casi idéntica ($3{,}16\,\%$) pero agota el tiempo máximo de $3600\,\text{s}$.
- A lo largo de todas las entregas, en instancias como `GCP_13` y `GCP_19` ningún método logró quebrar la barrera del $17\,\%$ de GAP en estos grafos, estableciendo el límite de las estrategias de búsqueda exploradas.
- El diseño de los operadores (cruce, mutación, reparación y vecindarios) aseguró un $100\,\%$ de soluciones factibles durante la totalidad del proyecto sin importar la agresividad de las perturbaciones.

REFERENCIAS

- J. C. Rivera Agudelo, "Diapositivas de clase del curso Heurística," Universidad EAFIT, 2026.
- Genetic and hybrid algorithms for graph coloring.
  https://link.springer.com/article/10.1007/BF02125407
- A general variable neighborhood search approach for the minimum load coloring problem.
  https://link.springer.com/article/10.1007/s11590-022-01861-1
- Genetic Algorithm.
  https://www.sciencedirect.com/topics/engineering/genetic-algorithm